



SFML

... para o TP3

INF110 – Programação I

Prof. André Gustavo
DPI/UFV – 2026/1





SFML



- Simple and Fast Multimedia Library
- Biblioteca para desenvolvimento de jogos 2D
- Tutorial baseado em:
 - <https://www.sfml-dev.org/tutorials/3.1/>
- Os programas usados no tutorial estão disponíveis no PVANet



Instalação

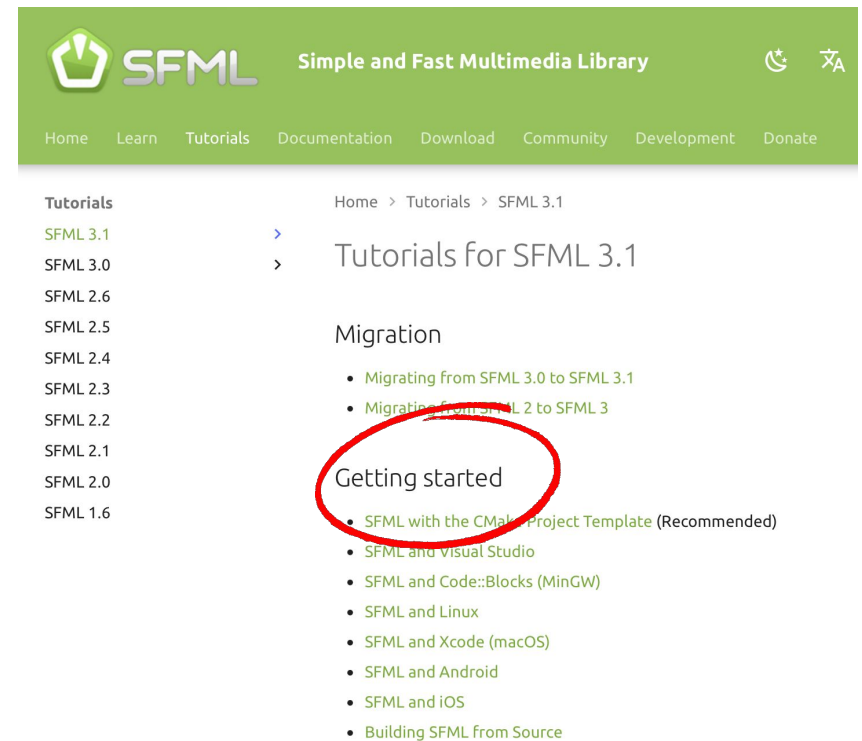
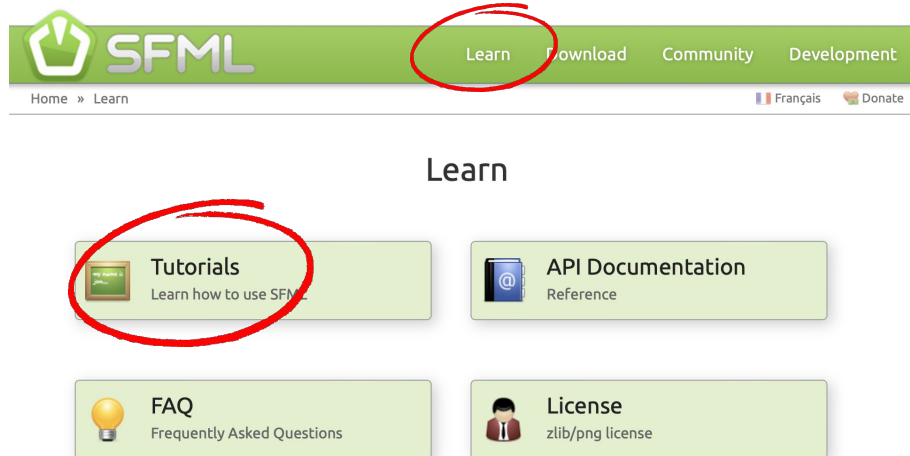
- Baixe a versão compatível com seu sistema operacional

The screenshot shows the SFML website with the following elements:

- Header:** SFML logo, "Simple and Fast Multimedia Library", search bar, and SFML version 3.1.0 with 11.9k stars and 1.9k forks.
- Navigation:** Home, Learn, Tutorials, Documentation, **Download** (circled in red), Community, Development, Donate.
- Left Sidebar:** Download, SFML 3.1.0, SFML 3.0.2, SFML 3.0.1, SFML 3.0.0, SFML 2.6.2, Older Versions, Bindings, Goodies.
- Main Content:** "Download" heading, "Home > Download" breadcrumb, and a grid of links:
 - Latest stable version:** SFML 3.1.0 (circled in red)
 - Snapshots:** In development versions
 - Bindings:** SFML in other languages
 - Git repository:** GitHub.com
 - Goodies:** Logos
 - Older versions:** SFML 1.6 and 2.x

+ Instalação

- Instale conforme instruções





Compilação



- SFML contém 5 módulos:
 - system, window, graphics, network e audio
- Existe uma biblioteca para cada um deles
- Para gerar o executável, é preciso linkar as bibliotecas usadas
- Para isso, adicione "-lsfml-xxx" na linha de comando, p.ex.

```
g++ prog.cpp -lsfml-graphics -lsfml-window -lsfml-system
```

- Ou configure a IDE conforme instruções do site

+ Teste de instalação

- Instale a SFML
- Compile o programa **0-teste.cpp**

```
g++ 0-teste.cpp -lsfml-graphics -lsfml-window -lsfml-system
```

- Execute o programa e deverá obter uma janela como a abaixo





Window



- Janelas na SFML são definidas pela classe `sf::Window`
- Quando uma janela é criada, ela é aberta:

```
#include <SFML/Window.hpp>

int main()
{
    sf::Window window(sf::VideoMode({800, 600}), "My window");
    ...
}
```



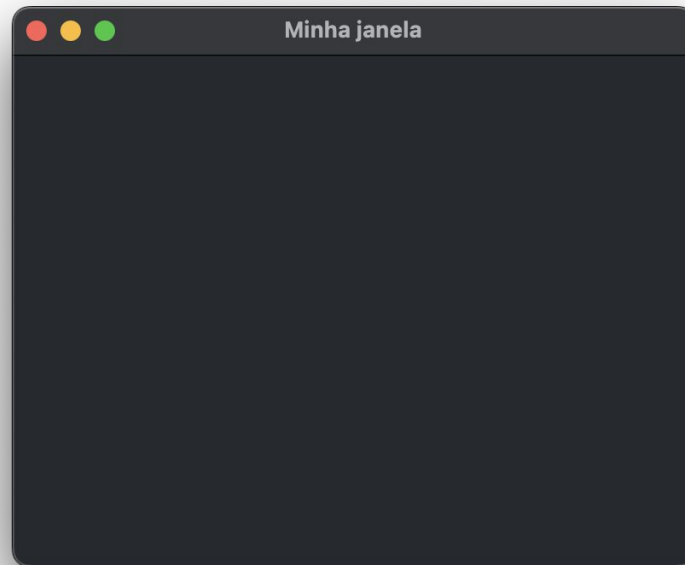
- Nesse exemplo, 800x600 é o tamanho e "My window", o título



Window – **1-window.cpp**



- Compile e execute o programa **1-window.cpp**
- Ele abre uma janela, que fica aberta até ser fechada pelo usuário





Window – 1-window.cpp

```
#include <SFML/Window.hpp>
```

```
int main()
```

```
{
```

```
    sf::Window window(sf::VideoMode({800, 600}), "My window");
```

```
    // run the program as long as the window is open
```

```
    while (window.isOpen())
```

```
    {
```

```
        // check all the window's events that were triggered since the last iteration of the loop
```

```
        while (const std::optional event = window.pollEvent())
```

```
        {
```

```
            // "close requested" event: we close the window
```

```
            if (event->is<sf::Event::Closed>())
```

```
                window.close();
```

```
        }
```

```
    }
```

```
}
```

Loop principal (todo programa tem isso):
fica atualizando a aplicação até que a
janela seja fechada

Tratamento de eventos: (1ª coisa a se fazer)
verificar eventos pendentes (fechou janela?
pressionou tecla? moveu o mouse? ...)
e tratar cada um deles

Usuário "fechou" a janela: no caso, solicitou
fechamento; ela só é fechada mesmo no
window.close() – pode fazer algo antes,
p.ex., salvar conteúdo, exibir mensagem...



Window – 1-window.cpp



Tarefa:

- Escreva uma mensagem logo antes de fechar a janela

```
cout << "Fechou...\n";
```

- Note que a mensagem aparecerá no terminal



Event



- O programa fica em loop, redesenhando a tela e tratando eventos à medida que aparecem
- Eventos incluem:
 - ações do teclado (pressionar tecla, soltar tecla, ...)
 - ações do mouse (clicar, mover, entrar/sair na janela, ...)
 - ações do joystick (movimentar, ...)
 - ações na janela (fechar, redimensionar, perder/ganhar foco...)
 - ...

+ Event – 2-event.cpp

- Compile e execute o programa **2-event.cpp**
- Ele trata mais um evento: `sf::Event::KeyPressed`
- Este é o evento usado para tomar uma ação exatamente quando uma tecla é pressionada, por exemplo, mover um personagem, dar um salto com espaço, sair da tela com esc...
- Existe também o evento `sf::Event::KeyReleased`, que é acionado quando se solta uma tecla
- O código da tecla pode ser recuperado em `event.scancode` (código físico no teclado) ou `event.code` (código da tecla)

+ Event – 2-event.cpp



Tarefa:

- Modifique o programa para
 - escrever uma mensagem se for pressionada a tecla 'A'
 - outra mensagem se for a tecla seta para a direita
 - se for outra tecla, escrever seu código (code ou scancode)

Obs.: os códigos das teclas são definidos por um tipo enumerado; você pode usar `sf::Keyboard::ScanCode::A` para tecla 'A' e `sf::Keyboard::ScanCode::Right` para tecla seta para direita.

+ Event – **2b-event.cpp**

- Compile e execute o programa **2b-event.cpp**
- Ele mostra como recuperar o estado das teclas modificadoras (shift, alt, control, ...) quando uma tecla é pressionada

Tarefa:

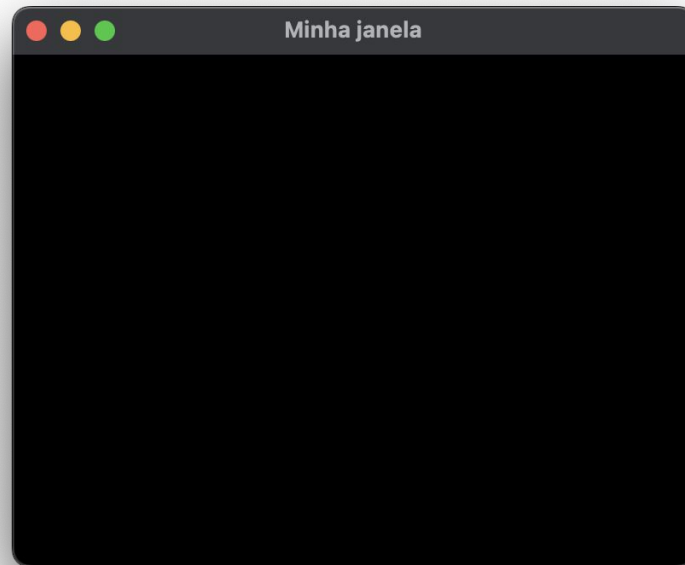
- Modifique o programa para fechar a janela quando se pressiona Shift + Esc



Draw window – **3-draw.cpp**



- Compile e execute o programa **3-draw.cpp**
- Ele cria uma janela de fundo preto





Draw window – 3-draw.cpp



- Parece o mesmo que o 1-window.cpp, mas é diferente
- A janela é `sf::RenderWindow`, não `sf::Window` de antes
- Ela tem tudo o que uma `sf::Window` tem e mais...
- Ela tem várias funcionalidades para desenhar coisas
- Nesse exemplo, temos a `clear`, que limpa a janela na cor escolhida, e a `display`, que de fato mostra a janela



Draw window – 3-draw.cpp



- O ciclo `clear/draw/display` é repetido continuamente
 - `clear`: limpa janela (apaga tudo)
 - `draw`: desenha as coisas (fundo, personagens, textos, etc)
 - `display`: mostra o que foi desenhado
- `clear` e `draw` são feitos em background, num buffer escondido; depois de tudo pronto, `display` copia do buffer para a janela



Draw window – 3-draw.cpp



- O ciclo `clear/draw/display` é a melhor forma de desenhar
- É assim que a tela de um jogo é atualizada!
 - Para mover um personagem, ele é apagado e redesenhado no novo local
 - Coisas que não se movem são apagadas e redesenhadas no mesmo local
- **Não** tente outras estratégias, como manter coisas do frame anterior, apagar algumas coisas, desenhar uma vez e display múltiplas vezes...
- Não tenha medo de desenhar 1000 coisas 60 vezes por segundo; isso é pouco comparado com milhões de triângulos que o computador suporta
- Placas gráficas *são projetadas* para esse ciclo: apagar tudo e desenhar tudo novamente em cada iteração do loop principal



Draw shape – 4-shape.cpp



- Compile e execute o programa **4-shape.cpp**
- Ele cria uma janela de fundo preto com um círculo verde





Draw shape – 4-shape.cpp



```
// cria um círculo de raio 50
sf::CircleShape shape(50.f);

// define a posição absoluta do círculo
shape.setPosition({10.f, 50.f});

// define a cor do círculo (verde)
shape.setFillColor(sf::Color(100, 250, 50));

// executa o programa enquanto a janela está aberta
while (window.isOpen()) {

    ...

    // limpa a janela com a cor preta
    window.clear(sf::Color::Black);

    // desenhar tudo aqui...

    // desenha o círculo na janela
    window.draw(shape);

    // termina e desenha o frame corrente
    window.display();
}
```

- shape círculo criado uma vez
- mas apagado e redesenhado em toda iteração

clear: limpa tudo

draw: desenha coisas

display: mostra tudo

loop principal



Draw shape – 4-shape.cpp



- Tarefas:
 - adicionar um círculo amarelo exatamente no centro da janela



Draw shape – 4-shape.cpp



■ Tarefas:

- adicionar um círculo amarelo exatamente no centro da janela
- adicionar um retângulo 100 x 50 em cada canto

```
// define um retângulo 100x50  
sf::RectangleShape rectangle({100.f, 50.f});
```

exemplo do RectangleShape

obs.: você pode declarar rectangle2, rectangle3, ... ou, de forma mais simples, usar somente o rectangle e dentro do loop principal colocar vários setposition seguidos de window.draw(rectangle); cada execução do draw desenha um retângulo.



Draw shape – 4-shape.cpp



■ Tarefas:

- adicionar um círculo amarelo exatamente no centro da janela
- adicionar um retângulo 100 x 50 em cada canto

```
// define um retângulo 100x50  
sf::RectangleShape rectangle({100.f, 50.f});
```

exemplo do RectangleShape

obs.: você pode declarar rectangle2, rectangle3, ... ou, de forma mais simples, usar somente o rectangle e dentro do loop principal colocar vários setposition seguidos de window.draw(rectangle); cada execução do draw desenha um retângulo.

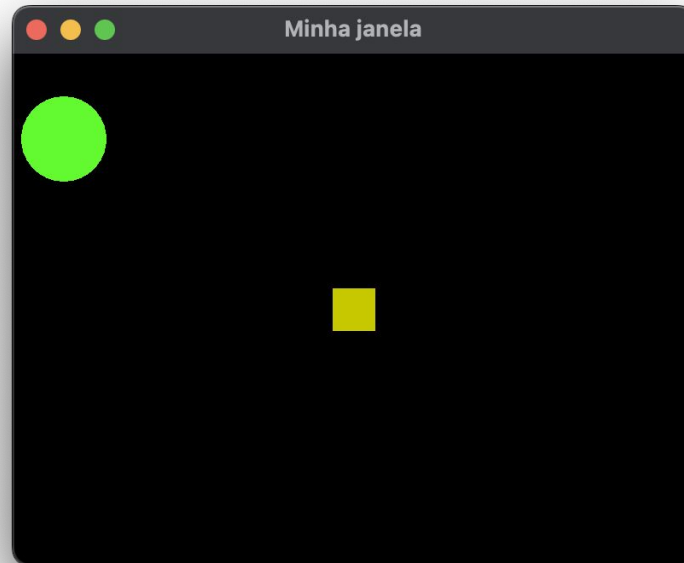
- mudar a cor do círculo aleatoriamente (a cada frame)



Draw shape – 4b-shape.cpp



- Compile e execute o programa **4b-shape.cpp**
- Ele adiciona um quadrado amarelo no centro
- O quadrado pode ser movimentado com as setas esquerda/direita!





Draw shape – 4b-shape.cpp

```
// cria um quadrado de tamanho 50
sf::RectangleShape quad({50.f, 50.f});
quad.setFillColor(sf::Color(200, 200, 00));
```

cria um shape
quadrado

```
float posx = 375, posy = 275; //posicao do quadrado
```

```
// executa o programa enquanto a janela está aberta
while (window.isOpen()) {
```

```
    // verifica todos os eventos que foram acionados na janela desde a última iteração do loop
```

```
    while (const std::optional event = window.pollEvent()) {
```

```
        // evento "fechar" acionado: fecha a janela
```

```
        if (event->is<sf::Event::Closed>())
```

```
            window.close();
```

```
        else if (const auto* keyPressed = event->getIf<sf::Event::KeyPressed>())
```

```
            if (keyPressed->scancode == sf::Keyboard::ScanCode::Escape)
```

```
                window.close();
```

```
            else if (keyPressed->scancode == sf::Keyboard::ScanCode::Left)
```

```
                posx -= 10; // left key: move o quadrado para esquerda
```

```
            else if (keyPressed->scancode == sf::Keyboard::ScanCode::Right)
```

```
                posx += 10; // right key: move o quadrado para direita
```

```
        }
```

```
    }
```

```
// limpa a janela com a cor preta
```

```
window.clear(sf::Color::Black);
```

```
// desenha o círculo na janela
```

```
window.draw(circ);
```

```
// reposiciona o quadrado
```

```
quad.setPosition({posx, posy});
```

```
// desenha o quadrado na janela
```

```
window.draw(quad);
```

```
// termina e desenha o frame corrente
```

```
window.display();
```

```
}
```

altera o valor da
variável de posição

clear: limpa tudo

draw: desenha o quadrado
(em nova posição)

display: mostra tudo



Draw shape – 4b-shape.cpp



- Note que "movimentar" o quadrado consiste em redesenhá-lo em nova posição no ciclo clear/draw/display
- Quando uma seta é pressionada, imediatamente muda-se o valor da variável que indica onde o quadrado é desenhado
- Mas a "movimentação" em si só é feita quando ele é redesenhado (*atraso imperceptível, a janela é redesenhada várias vezes por seg.*)



Draw shape – 4b-shape.cpp



■ Tarefas:

- movimentar o quadrado também para cima e para baixo

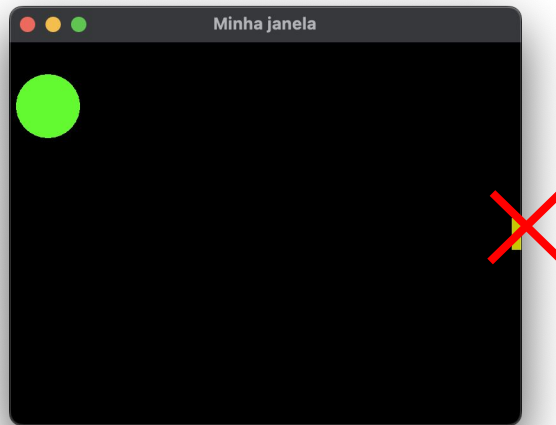


Draw shape – 4b-shape.cpp



■ Tarefas:

- movimentar o quadrado também para cima e para baixo
- impedir que o quadrado saia da janela





Draw shape – 4b-shape.cpp



■ Tarefas:

- movimentar o quadrado também para cima e para baixo
- impedir que o quadrado saia da janela
- em vez de impedir que saia da janela, fazê-lo aparecer do outro lado

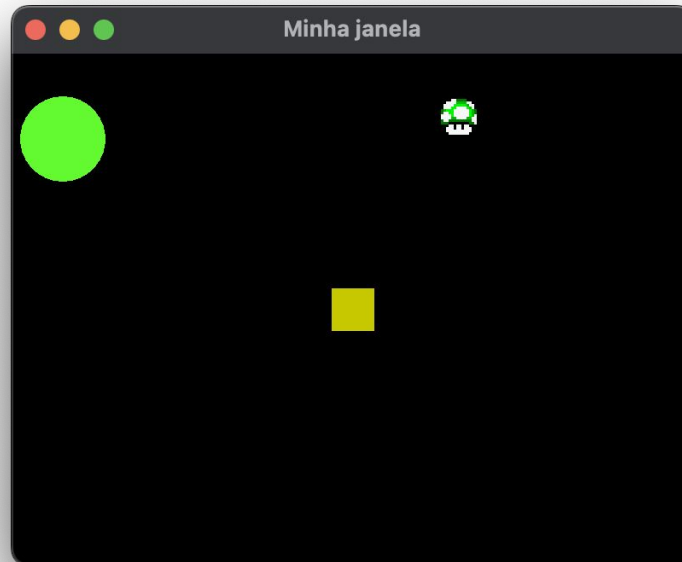




Draw texture/sprite – 5-sprite.cpp



- Compile e execute o programa **5-sprite.cpp**
- Ele adiciona uma "vida" na janela





Draw texture/sprite – 5-sprite.cpp

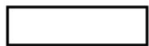


```
// cria uma "textura" (uma figura)
sf::Texture texture;
if (!texture.loadFromFile("1up.png")) {
    std::cout << "Erro lendo imagem 1up.png\n";
    return 0;
}
// cria um sprite (um objeto gráfico) com a figura
sf::Sprite sprite(texture);
sprite.setPosition({500, 50});
```

cria uma "texture"
com uma imagem

cria um sprite com a
texture (imagem)

- Sprite é uma caixa com uma textura (uma figura)



+



=



Rectangular entity

Texture

Sprite!



Draw texture/sprite – 5-sprite.cpp



```
// executa o programa enquanto a janela está aberta
while (window.isOpen()) {

    ...

    // limpa a janela com a cor preta
    window.clear(sf::Color::Black);

    // desenhar tudo aqui...

    // desenha o círculo na janela
    window.draw(circ);

    // reposiciona o quadrado
    quad.setPosition({posx, posy});
    // desenha o quadrado na janela
    window.draw(quad);
    // desenha sprite de vida na janela
    window.draw(sprite);

    // termina e desenha o frame corrente
    window.display();
}
```

sprites são desenhados da
mesma forma que shapes



Draw texture/sprite – 5-sprite.cpp



- **Tarefa:**
 - acrescentar outro sprite (um personagem qualquer à sua escolha)
 - acrescentar uma imagem de fundo



Draw texture/sprite – 5-sprite.cpp



■ Tarefa:

- acrescentar outro sprite (um personagem qualquer à sua escolha)
- acrescentar uma imagem de fundo
- criar um sprite com apenas parte de uma imagem
 - ao ler uma imagem, pode-se criar uma textura com parte dela
 - ex.: textura 32x32 píxel que começa na posição (10,10)

```
// cria uma textura tamanho 32x32 começando na posição (10, 10) da imagem
sf::Texture texture;
if (!texture.loadFromFile("image.png", false, sf::IntRect({10, 10}, {32, 32})))
{
    // error...
}
```

obs.: bastante útil pois há imagens disponíveis na web contendo vários personagens, em diferentes posições

+ Time – 6-time.cpp

- Compile e execute o programa **6-time.cpp**
- Ele movimenta o sprite vida para a direita automaticamente





Time – 6-time.cpp



- O tempo decorrido pode ser medido com a classe `sf::Clock`
- Ela tem apenas duas funções
 - `getElapsedTime`: tempo decorrido desde que o relógio começou
 - `restart`: reinicia o relógio
- No programa, é usado para movimentar o sprite vida a cada 0.1s

```
// cria um relógio para medir o tempo  
sf::Clock clock;
```

```
// Muda a posição do sprite vida a cada 0.1 segundos  
if (clock.getElapsedTime() > sf::seconds(0.1)) {  
    clock.restart();  
    posxup += 10;  
}
```

passou mais de 0,1 s?

muda posição e reinicia a contagem de tempo



Time – 6-time.cpp



- Para marcar intervalos de tempo, usa-se a classe `sf::Time`
- Os valores representam uma certa quantidade de tempo, mas não em unidades de tempo propriamente, como segundos, minutos, etc
- Existem funções para converter unidades de tempo específicas
 - `sf::seconds(0.1f)` no if do programa significa 0,1 segundos



Time – 6-time.cpp



- Para marcar intervalos de tempo, usa-se a classe `sf::Time`
- Os valores representam uma certa quantidade de tempo, mas não em unidades de tempo propriamente, como segundos, minutos, etc
- Existem funções para converter unidades de tempo específicas
 - `sf::seconds(0.1f)` no if do programa significa 0,1 segundos
 - O mesmo valor pode ser obtido por
`sf::milliseconds(100)` ou `sf::microseconds(100000)`



Time – 6-time.cpp



- Para marcar intervalos de tempo, usa-se a classe `sf::Time`
- Os valores representam uma certa quantidade de tempo, mas não em unidades de tempo propriamente, como segundos, minutos, etc
- Existem funções para converter unidades de tempo específicas
 - `sf::seconds(0.1f)` no if do programa significa 0,1 segundos
 - O mesmo valor pode ser obtido por `sf::milliseconds(100)` ou `sf::microseconds(100000)`
- Pode-se criar uma variável para armazenar o tempo e usá-la depois

```
// tempo de 0.1 segundos na unidade da sf::Time
const sf::Time Tempo = sf::seconds(0.1);
```

```
// Muda a posição do sprite vida a cada 0.1 segundos
if (clock.getElapsedTime() > Tempo) {
    clock.restart();
    posxup += 10;
}
```

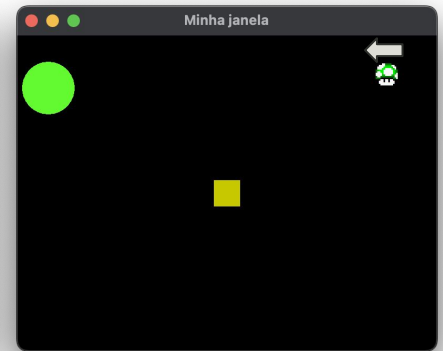
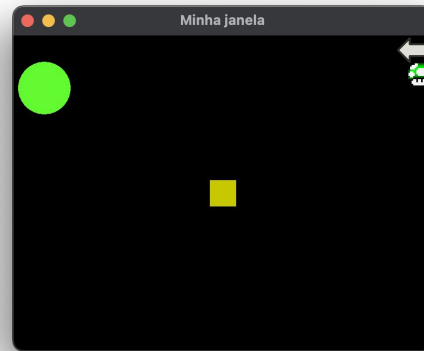
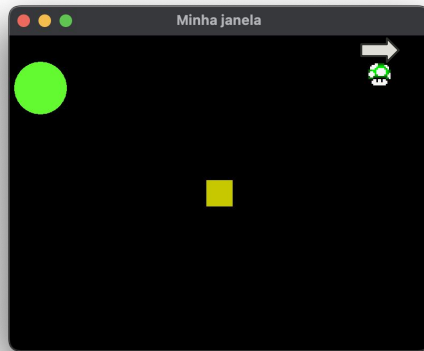
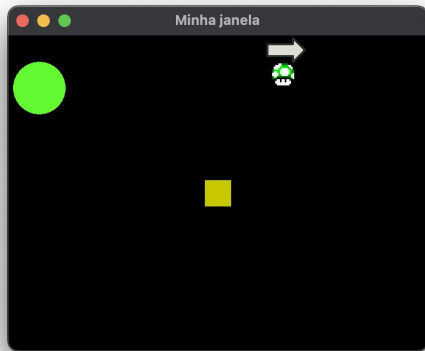
+ Time – 6-time.cpp

- Tarefa:
 - mover o sprite vida na mesma velocidade, mas de forma mais "suave"

+ Time – 6-time.cpp

■ Tarefa:

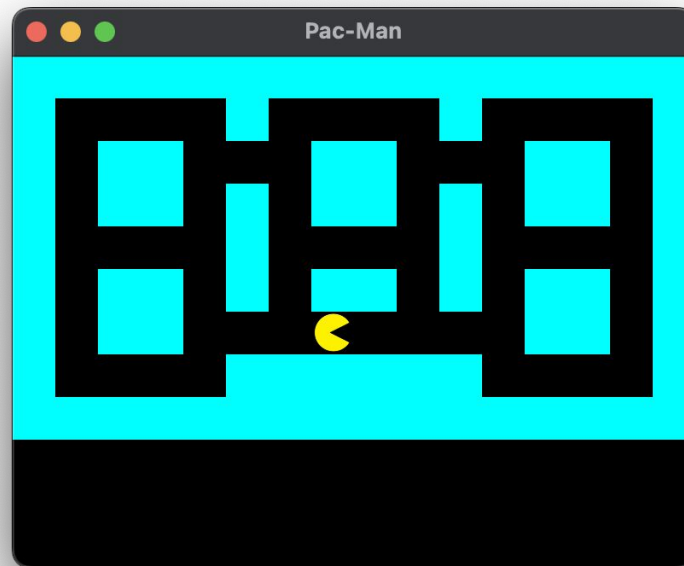
- mover o sprite vida na mesma velocidade, mas de forma mais "suave"
- não deixar a vida escapar da tela!
quando atingir o limite da direita, passar a movimentá-la para a esquerda





Juntando tudo! – 7-pacman.cpp

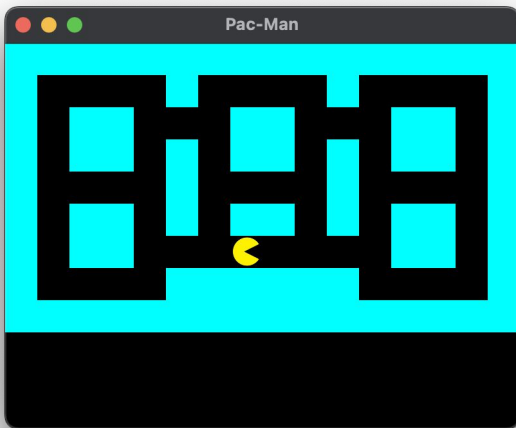
- Compile e execute o programa **7-pacman.cpp**
- Ele abre um cenário (tosco e incompleto) de Pac-Man...





Juntando tudo! – 7-pacman.cpp

- Note que o mapa não é uma figura de fundo...
- Ele é desenhado a partir de uma matriz
- 1 = parede, 0 = vazio
- cada "pixel" da matriz é um quadrado 50x50



```
char mapa[9][17] = {  
    "1111111111111111",  
    "1000010000100001",  
    "1011000110001101",  
    "1011010110101101",  
    "1000010000100001",  
    "1011010110101101",  
    "1011000000001101",  
    "1000011111100001",  
    "1111111111111111"  
};
```

```
// parede  
sf::RectangleShape quad({50.f, 50.f});  
quad.setFillColor(sf::Color(0, 100, 200));
```

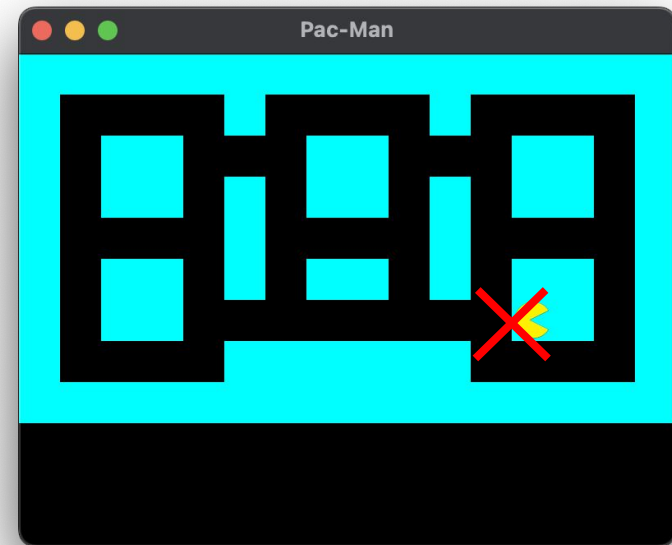
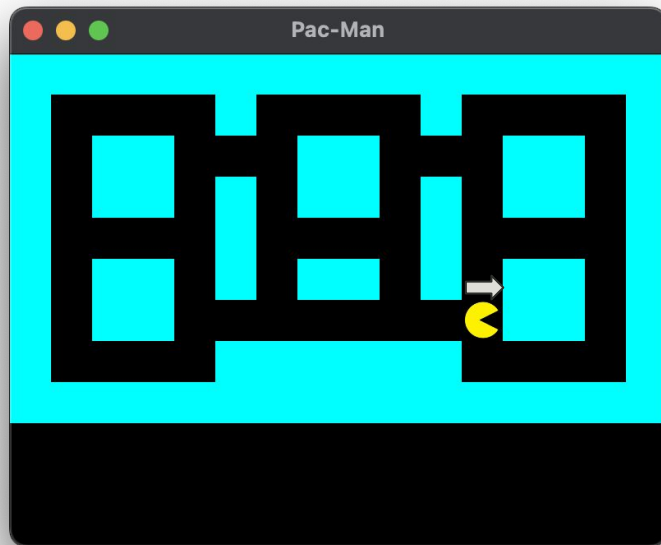
```
// desenha paredes  
for(int i=0; i<9; i++)  
    for(int j=0; j<17; j++)  
        if (mapa[i][j]=='1') {  
            quad.setPosition({j*50.f, i*50.f});  
            window.draw(quad);  
        }
```



Juntando tudo! – 7-pacman.cpp

- Tarefa:

- não deixar o Pac-Man atravessar as paredes!

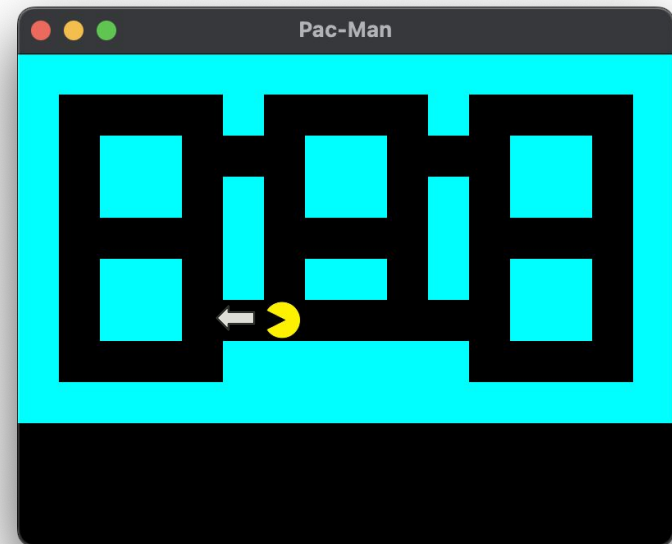
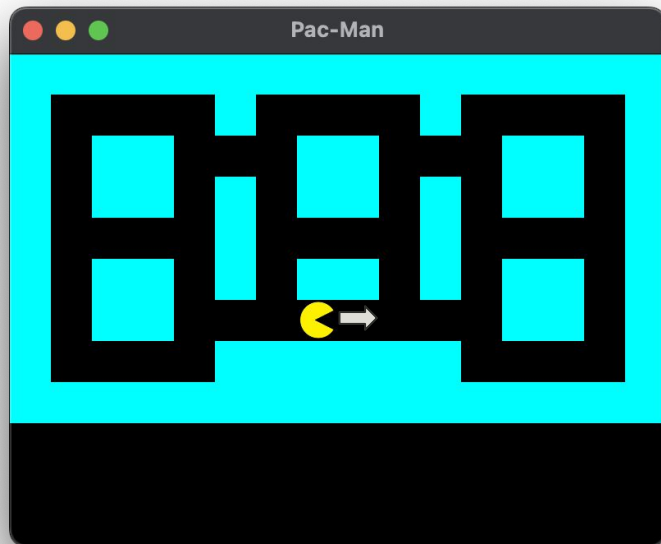




Juntando tudo! – 7-pacman.cpp

- Tarefa:

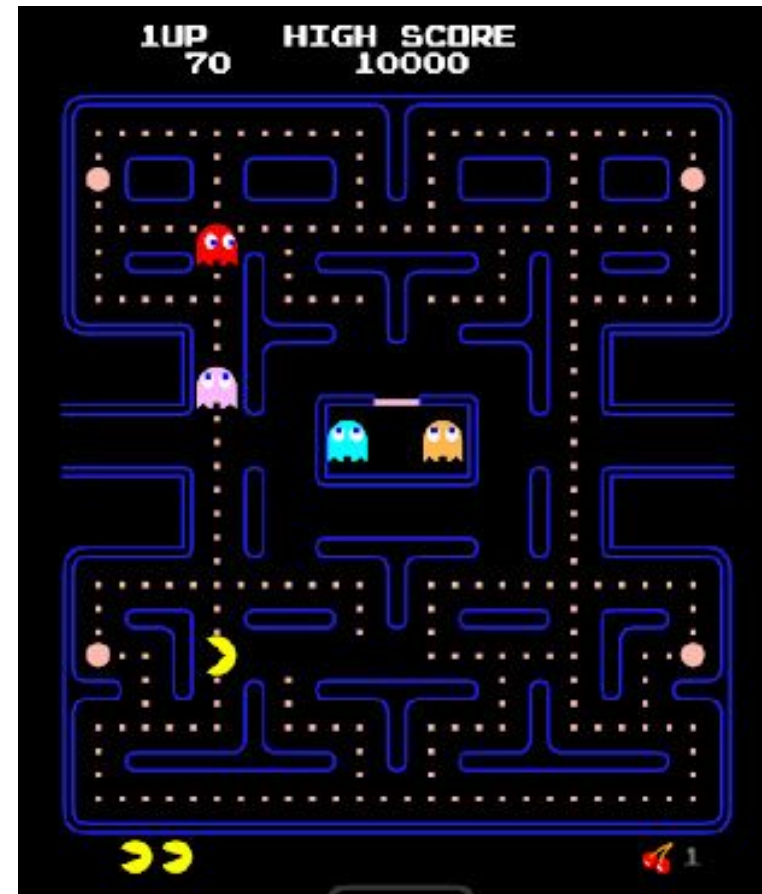
- mudar a figura do PacMan quando ele se move para a esquerda





Pac-man – Trabalho final

- PacMan controlado pelo teclado
 - Vence ao comer todas as pílulas
 - Morre se encostar em um fantasma

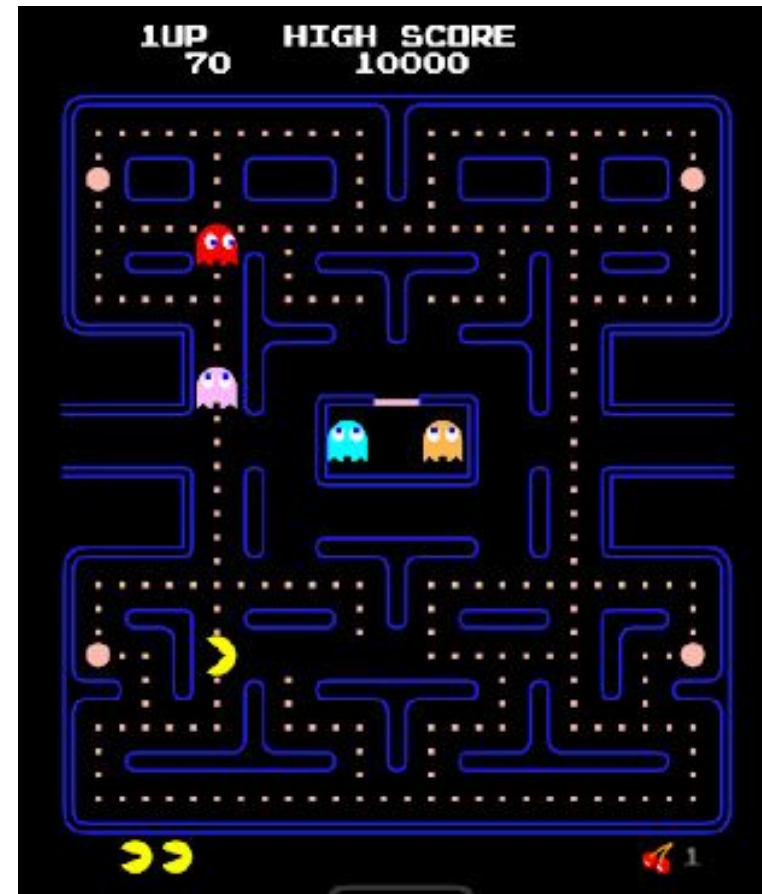




Pac-man – Trabalho final

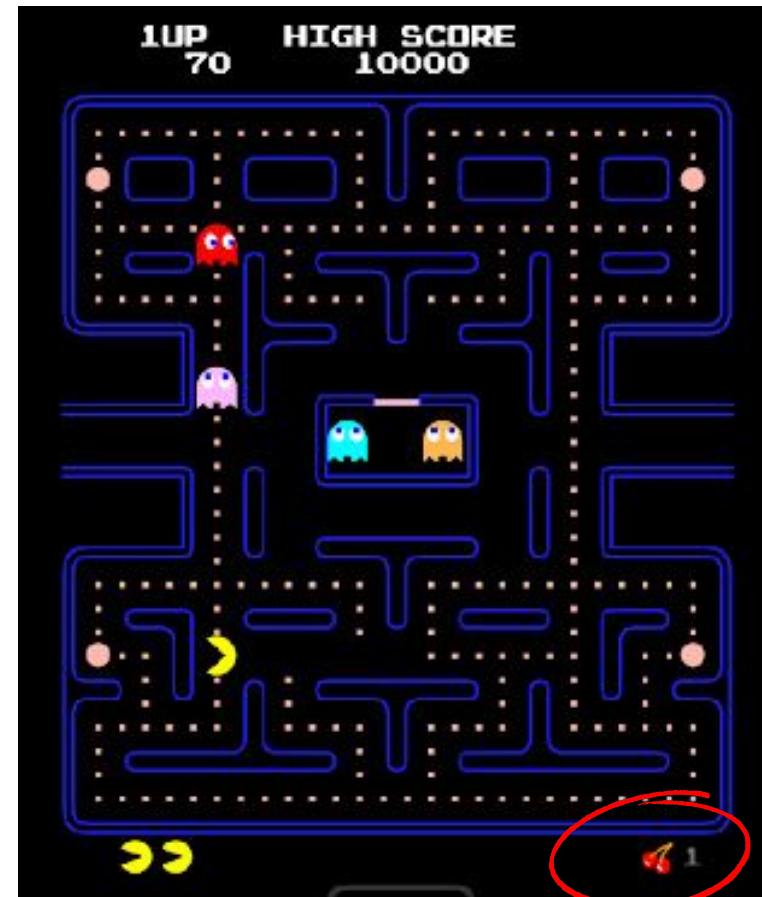


- PacMan controlado pelo teclado
 - Vence ao comer todas as pílulas
 - Morre se encostar em um fantasma
- Fantasmas se movem sozinhos
 - Mudam de direção numa encruzilhada
 - Alguns aleatoriamente
 - Outros "perseguem" o PacMan



+ Pac-man – Trabalho final

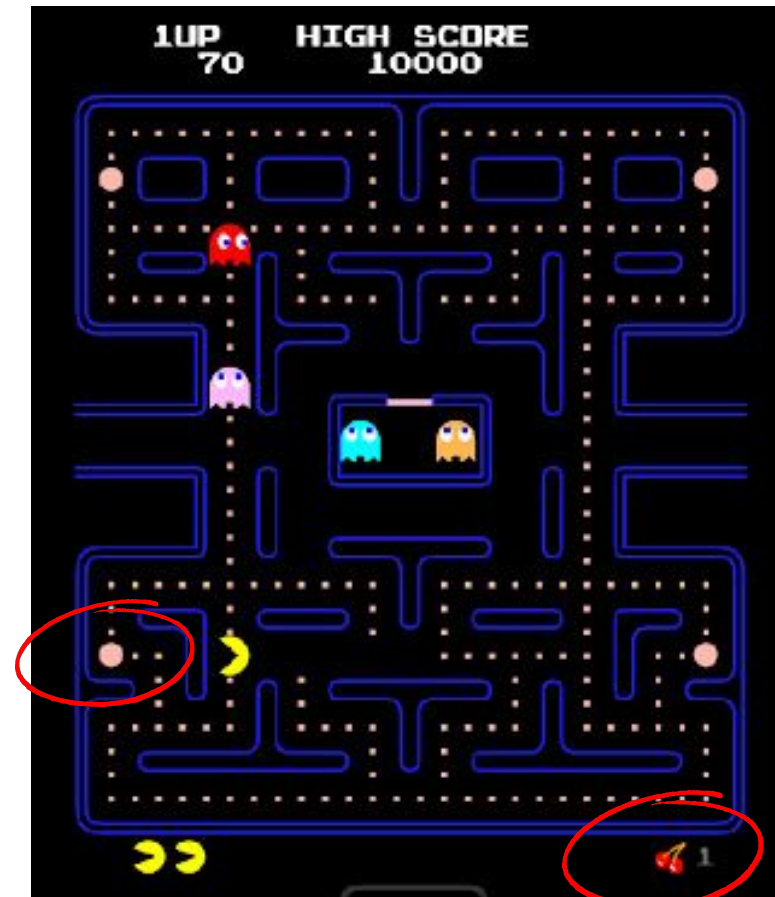
- Não precisa ter as frutinhas





Pac-man – Trabalho final

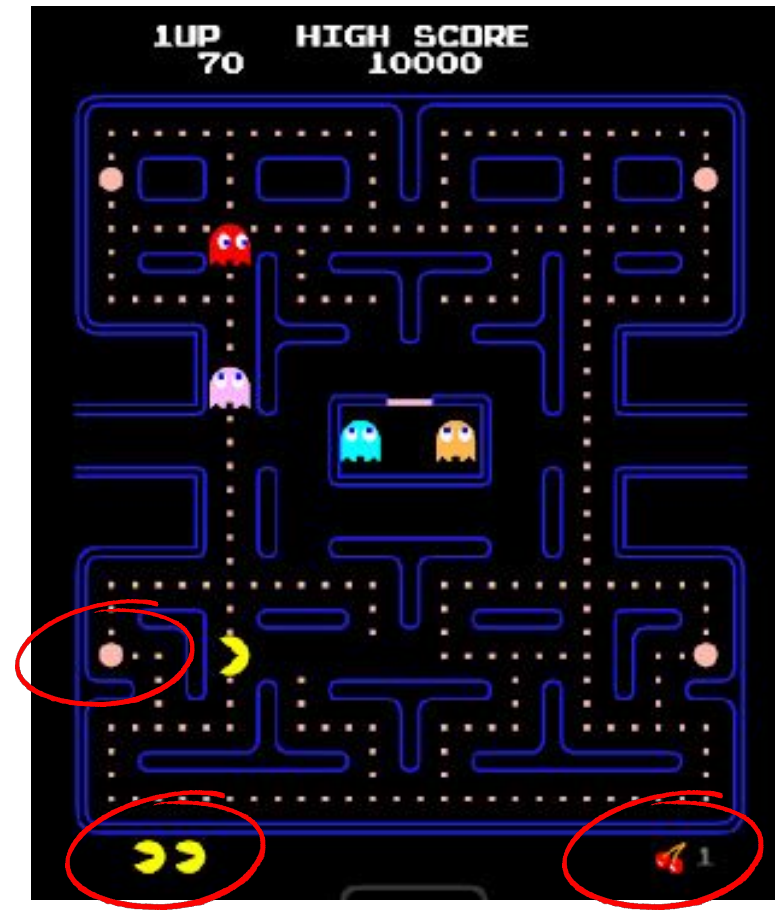
- Não precisa ter as frutinhas
- Não precisa ter as pílulas de força





Pac-man – Trabalho final

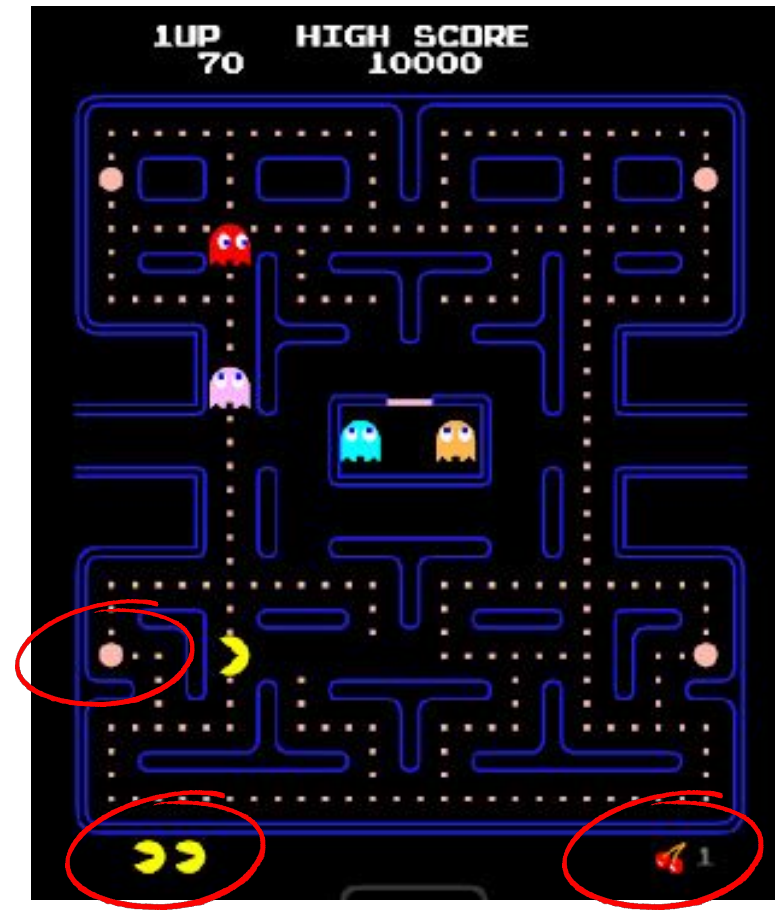
- Não precisa ter as frutinhas
- Não precisa ter as pílulas de força
- Não precisa ter vidas





Pac-man – Trabalho final

- Não precisa ter as frutinhas
- Não precisa ter as pílulas de força
- Não precisa ter vidas
- Não precisa ser o mapa original
- Nem os personagens originais
(pode criar os seus! ou um "crossover")





Mais...



■ Outras funcionalidades

Graphics ▼

Drawing 2D stuff

Sprites and textures

Text and fonts

Shapes

Audio ▼

Playing sounds and music

Recording audio

Custom audio streams

Spatialization: Sounds in 3D



Movimento contínuo – **7b-pacman.cpp**



- Sugestão para um movimento mais suave, contínuo, sem necessidade de pressionar uma tecla repetidas vezes:
 - usar booleanos, ativados quando a tecla é pressionada (e desativados quando é liberada)
 - movimentar pelo status dos booleanos
- Ver **7b-pacman.cpp**
 - nesse código, o usuário não precisa pressionar a seta o tempo todo
 - o PacMan continua se movendo na última direção informada



Movimento contínuo – 7b-pacman.cpp



- Sugestão para um movimento mais suave, contínuo, sem necessidade de pressionar uma tecla repetidas vezes:
 - usar booleanos, ativados quando a tecla é pressionada (e desativados quando é liberada)
 - movimentar pelo status dos booleanos
- Ver **7b-pacman.cpp**
 - nesse código, o usuário não precisa pressionar a seta o tempo todo
 - o PacMan continua se movendo na última direção informada
- **Tarefa:**
 - adicionar opção de reiniciar: tecla 'R' reposiciona na posição inicial